

HKPUG Meetup | June 2026

L.E.P.A.U.T.E. Framework: FSD-style Visual Perception with Lie Group Equivariant Attention

Building robust spatial understanding in Python

Speaker: Carson Wu

CV & NLP Developer

Hong Kong Python User Group

27 June 2026

Speaker & Project



Scan to Connect



L.E.P.A.U.T.E. Repository

Note: This is an independent technical project. No affiliation with or endorsement by Tesla.

The Challenge of Spatial Perception

Autonomous driving perception (FSD-style) requires more than 2D object detection.

- Understand **rotation**, **translation**, and **viewpoint changes**.
- Maintain consistent scene understanding despite camera motion.
- Reason about **scale**, **depth**, and **relative pose**.

Standard CNNs struggle here because they treat images as flat pixel grids without built-in geometry.

Why Lie Groups?

Lie Groups provide the mathematical language for continuous rigid motions.

- **SE(3)**: Special Euclidean group in 3D: rotations + translations.
- Smooth manifold + group structure.
- **Lie Algebra**

$$\mathfrak{se}(3)$$

: Flat 6D tangent space (3 translation + 3 rotation twist).

Key advantage: All operations stay geometrically valid. No invalid poses.

Core Mathematical Tools

Implemented in `module.py`

- **Skew-symmetric matrix:**

$$[\omega]_{\times} =$$

`skew_symmetric(omega)`

- **Exponential Map:**

$$\exp : \mathfrak{se}(3) \rightarrow \text{SE}(3)$$

`(se3_exp_map)`

$$T = \begin{pmatrix} R & t \\ 0 & 1 \end{pmatrix}$$

Closed-form using Rodrigues formula for rotation and translation.

- **Logarithmic Map:**

$$\log : \text{SE}(3) \rightarrow \mathfrak{se}(3)$$

(se3_log_map)

- **Composition:**

$$T_1 \circ T_2 = T_1 T_2$$

(compose_poses)

These maps are fully differentiable -> end-to-end training.

Lie Groups vs. Standard CNNs

Standard CNNs on raw 4x4 matrices or Euler angles fail because:

- Gradients distort rotation matrices (lose orthogonality).
- Gimbal lock and discontinuities in Euler angles.
- Must learn geometry implicitly from millions of examples (“raw materials”).

Why Lie Groups win:

- Enforce geometric constraints by design (equivariance: $f(gc \cdot x) = gc \cdot f(x)$).
- Work with far less data.
- Gradients respect manifold structure.
- Robust under viewpoint changes, rotations, and translations.

Why CNNs Need Massive Raw Data

A plain CNN predicting poses must rediscover physics from scratch:

- How pixels change under rotation/translation.
- Parallax effects from depth.
- Rigidity and continuity constraints.

This requires enormous diverse datasets (lighting, textures, motions, viewpoints) to avoid overfitting and physically impossible outputs.

Lie groups inject these rules mathematically. The network learns **residuals** only, not the entire geometry -> better generalization with smaller datasets.

L.E.P.A.U.T.E. Architecture

Lightweight solo-developed framework combining:

- Lie group geometry.
- Asynchronous multi-threaded pipeline.
- Hybrid tracking (direct + neural + kinematic recovery).
- Zero-shot open-vocabulary classification (SigLIP).

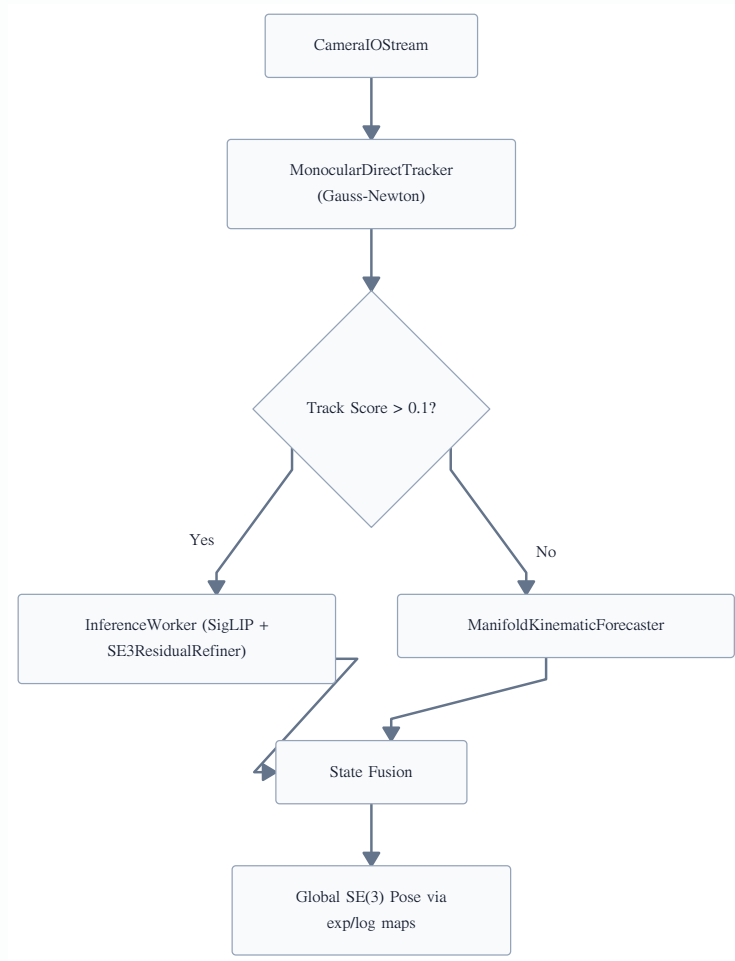
Asynchronous Pipeline Design

Mixing heavy inference with capture causes collapse under load.

- **CameraIOStream**: Isolated ingestion (real camera or mock).
- **InferenceWorker**: Dedicated thread + bounded queue + lock-protected history buffer.
- **ManifoldKinematicForecaster**: SE(3) velocity-based recovery when neural results lag.
- **SequenceDataCollector**: Async SQLite WAL logging.

This concurrent design keeps the system stable.

Hybrid Tracking Pipeline



Differentiable SE(3) warping (`MonocularSE3Warping`) enables direct photometric gradients.

Equivariant Attention & Residual Refiner

- SE3ResidualRefiner: Learns corrections in Lie algebra space.
- Heteroscedastic loss:

$$0.5 \exp(-\text{unc}) \|\Delta\xi\|^2 + 0.5 \text{ unc}$$

- Equivariant design ensures consistent understanding under transformations.

Open-Vocabulary & Training

- SigLIP for runtime object labels and scale priors.
- Dataset from BOP (YCB-V) via relative SE(3) computation.
- Training script: `train.py` with PyTorch compile support.

Defensive Design & Lessons

- Geometry provides robustness with less data.
- Async isolation prevents pipeline collapse.
- Sensor dropout -> manifold forecasting.
- Memory safety via tensor detachment and WAL logging.

Hands-On and Live Demo

Real-time SE(3) Spatial Vision Pipeline

Q&A

Questions welcome on math, architecture, or implementation details.

Thank You!